Documentación técnica

Guía paso a paso de cómo se construyó el proyecto: comandos, resultado esperado de cada paso y capturas. Sirve para reproducirlo y para poder explicar cada decisión.

- Los identificadores de la suscripción están tapados en las capturas y sustituidos por `<ID-DE-SUSCRIPCION>` en los textos.
- Lo que **no** se ha verificado todavía está en `limitaciones.md` (docs/limitaciones.md).
- Los problemas que aparecieron por el camino están en `hallazgos.md` (docs/hallazgos.md).

Índice

1. Convenciones
2. Entorno
3. Protección del repositorio
4. Instalación de herramientas
5. Código de Terraform
6. Revisión de seguridad con Checkov
7. Conexión con Azure
8. Plan y despliegue de la red
9. Azure Policy
10. RBAC de mínimo privilegio
11. Lista de comprobaciones
12. Limpieza
13. Problemas encontrados
14. Índice de capturas

1. Convenciones

Convención	Detalle
Escritura de ficheros	`tee` con heredoc. `tee -a` **añade** al final y `tee` sin `-a` **sustituye** el fichero
Shell	El equipo usa fish, que no admite heredoc (`<<`). Antes de esos bloques se abre `bash`
Rutas	Relativas a la raíz del repositorio, salvo que se indique otra cosa
Terraform	Los comandos de Terraform se ejecutan desde `terraform/secure/`

2. Entorno

Elemento	Versión
Sistema	Ubuntu 24.04.5 LTS (`noble`)
Terraform	1.17.0
Proveedor `azurerm`	4.81.0
Azure CLI	2.91.0
Checkov	3.3.30 (instalado con `pipx`, Python 3.12.3)
Suscripción	Azure for Students, con rol de Propietario sobre la suscripción
Región	`germanywestcentral`

3. Protección del repositorio

Antes de ejecutar nada de Terraform se comprobó que el estado quedaba protegido. El repositorio **no tenía** `.gitignore`, así que se creó:

```
tee .gitignore > /dev/null << 'EOF'
# Estado de Terraform: contiene IDs de suscripción y de todos los recursos.
*.tfstate
*.tfstate.*

# Carpeta que crea "terraform init"
.terraform/
```

```
# Variables con valores reales
*.tfvars
*.tfvars.json

# Planes guardados
*.tfplan
tfplan

# Ficheros locales y registros de fallos
crash.log
crash.*.log
override.tf
override.tf.json
*_override.tf
*_override.tf.json
.terraformrc
terraform.rc

# Red de seguridad: claves y secretos típicos
.env
*.pem
*.key
EOF
```

El fichero `.terraform.lock.hcl` **no** se ignora a propósito: fija la versión del proveedor y conviene subirlo.

Comprobación. Cada ruta sensible debe estar cubierta por una regla:

```
git check-ignore -v terraform/secure/terraform.tfstate terraform/secure/prod.tfvars terraform/secure/.terraform/x
```

Resultado obtenido (una línea por ruta):

```
.gitignore:3:*.tfstate  terraform/secure/terraform.tfstate
.gitignore:10:*.tfvars  terraform/secure/prod.tfvars
.gitignore:7:.terraform/    terraform/secure/.terraform/x
```

Antes de arreglarlo, el mismo comando no devolvía nada (ver hallazgo 1 (docs/hallazgos.md)).

Hábito antes de cada commit: `git status --short` y `git add` con rutas concretas, nunca `git add .`.

4. Instalación de herramientas

4.1 Checkov

```
sudo apt install -y pipx
pipx install checkov
pipx ensurepath
```

Hay que abrir una terminal nueva para que se cargue el `PATH`. Se instaló la versión 3.3.30.

4.2 Terraform

Se añade el repositorio oficial de HashiCorp con su clave de firma, para que `apt` verifique los paquetes:

```
sudo apt update && sudo apt install -y gnupg curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
terraform version
```

La línea del repositorio debe terminar en `noble main`. Resultado:



Versión de Terraform

4.3 Azure CLI

Mismo método, con el repositorio de Microsoft:

```
sudo apt install -y apt-transport-https ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -sLS https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor | sudo tee /etc/apt/keyrings/microsoft.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/microsoft.gpg

sudo tee /etc/apt/sources.list.d/azure-cli.sources > /dev/null << 'EOF'
Types: deb
URIs: https://packages.microsoft.com/repos/azure-cli/
Suites: noble
Components: main
Architectures: amd64
Signed-by: /etc/apt/keyrings/microsoft.gpg
EOF

sudo apt update
sudo apt install -y azure-cli
az version
```

`noble` es el nombre de Ubuntu 24.04 y `amd64`, la arquitectura del equipo. El `apt update` **es imprescindible** tras añadir el repositorio (ver problemas). Salida de `az version` (extracto):

```
{
  "azure-cli": "2.91.0",
  "azure-cli-core": "2.91.0",
  "azure-cli-telemetry": "1.1.0",
  ...
```

5. Código de Terraform

Todo el código está en `terraform/secure/`. Cada fichero lleva comentarios en español. Aquí se resume qué hace cada uno.

5.1 `providers.tf`

Fija las versiones y desactiva el registro automático de servicios de Azure (ver hallazgo 2 (docs/hallazgos.md)):

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}

  # Por defecto Terraform intenta registrar decenas de servicios de Azure
  # antes de empezar, y tarda muchísimo. Aquí solo usamos red.
  resource_provider_registrations = "none"
}
```

El ID de suscripción **no** se escribe en ningún fichero: se pasa por la variable de entorno `ARM_SUBSCRIPTION_ID`.

5.2 `variables.tf`

Variable	Valor por defecto	Para qué
`location`	`germanywestcentral`	Región de los recursos
`proyecto`	`lz`	Prefijo para los nombres
`entorno`	`lab`	Nombre del entorno
`vnet_cidr`	`["10.10.0.0/16"]`	Rango de la red virtual
`subnet_mgmt_cidr`	`10.10.1.0/24`	Subred de gestión
`subnet_workload_cidr`	`10.10.2.0/24`	Subred de cargas de trabajo
`efecto_politica`	`Deny`	Efecto de la política (`Audit`, `Deny` o `Disabled`)

La variable `efecto_politica` valida su valor, de modo que un error tipográfico falla antes de tocar Azure:

```
variable "efecto_politica" {
  description = "Efecto de la política de IPs públicas: Audit (solo avisa), Deny (bloquea) o Disabled (apagada)"
  type        = string
  default     = "Deny"

  validation {
    condition     = contains(["Audit", "Deny", "Disabled"], var.efecto_politica)
    error_message = "El efecto debe ser Audit, Deny o Disabled."
  }
}
```

5.3 `network.tf`

Define ocho recursos. Los nombres se construyen con el prefijo `lz-lab`.

Recurso	Nombre resultante
Grupo de recursos	`rg-lz-lab`
Red virtual	`vnet-lz-lab` (`10.10.0.0/16`)
Subred de gestión	`snet-mgmt` (`10.10.1.0/24`)
Subred de cargas	`snet-workload` (`10.10.2.0/24`)
NSG de gestión	`nsg-mgmt-lz-lab`
NSG de cargas	`nsg-workload-lz-lab`
Asociación NSG-subred (x2)	Une cada firewall con su subred

Las asociaciones son imprescindibles: sin ellas las reglas existen pero no se aplican a nada.

La regla clave del NSG de cargas:

```
security_rule {
  name                       = "ssh-solo-desde-gestion"
  priority                   = 100
  direction                  = "Inbound"
  access                     = "Allow"
  protocol                   = "Tcp"
  source_port_range          = "*"
  destination_port_range     = "22"
  source_address_prefix      = var.subnet_mgmt_cidr
  destination_address_prefix = var.subnet_workload_cidr
}
```

Ambos NSG terminan con una regla `denegar-todo-entrante` de prioridad 4096. Todos los recursos llevan las etiquetas `proyecto`, `entorno` y `gestionado_por`.

5.4 `policy.tf`

Asigna al grupo de recursos la política integrada *Not allowed resource types*, prohibiendo el tipo `Microsoft.Network/publicIPAddresses` (extracto):

```
resource "azurerm_resource_group_policy_assignment" "sin_ip_publica" {
  name                 = "no-ip-publica"
  display_name         = "Prohibir IPs públicas en el lab"
```

```
  resource_group_id    = azurerm_resource_group.rg.id
  policy_definition_id = "/providers/Microsoft.Authorization/policyDefinitions/6c112d4e-5bc7-47ae-a041-ea2d9dccd749"

  parameters = jsonencode({
    effect = {
      value = var.efecto_politica
    }
    listOfResourceTypesNotAllowed = {
      value = ["Microsoft.Network/publicIPAddresses"]
    }
  })
}
```

5.5 `rbac.tf`

Tres recursos: un rol personalizado, una identidad administrada y la asignación entre ambos (extracto):

```
resource "azurerm_role_definition" "auditor_red" {
  name  = "lz-auditor-red"
  scope = azurerm_resource_group.rg.id

  permissions {
    actions = [
      "Microsoft.Resources/subscriptions/resourceGroups/read",
      "Microsoft.Network/virtualNetworks/read",
      "Microsoft.Network/virtualNetworks/subnets/read",
      "Microsoft.Network/networkSecurityGroups/read",
      "Microsoft.Network/networkSecurityGroups/securityRules/read",
    ]
    not_actions = []
  }

  assignable_scopes = [azurerm_resource_group.rg.id]
}

resource "azurerm_user_assigned_identity" "auditor" {
  name                = "id-auditor-${local.prefijo}"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  tags                = local.etiquetas
}

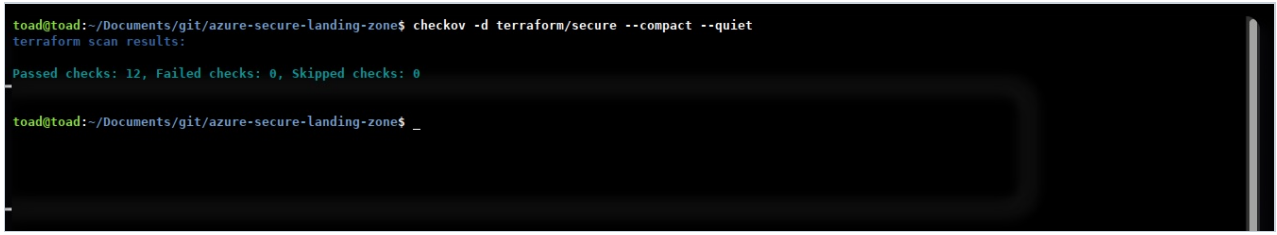
resource "azurerm_role_assignment" "auditor" {
  scope              = azurerm_resource_group.rg.id
  role_definition_id = azurerm_role_definition.auditor_red.role_definition_resource_id
  principal_id       = azurerm_user_assigned_identity.auditor.principal_id
  principal_type     = "ServicePrincipal"
}
```

6. Revisión de seguridad con Checkov

6.1 En local

```
checkov -d terraform/secure --compact --quiet
```

`--quiet` muestra solo los fallos y `--compact` quita el código de cada uno. Resultado sobre el código de red:



Checkov en local

Nota: este escaneo se hizo antes de añadir `policy.tf` y `rbac.tf`. Volver a ejecutarlo está en la lista de pendientes.

6.2 En GitHub Actions

El mismo escaneo se ejecuta en cada `push` y `pull request` hacia `main`:

```
name: Checkov - escaneo de seguridad IaC

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

# Mínimo privilegio: este pipeline solo puede LEER el repositorio
permissions:
  contents: read

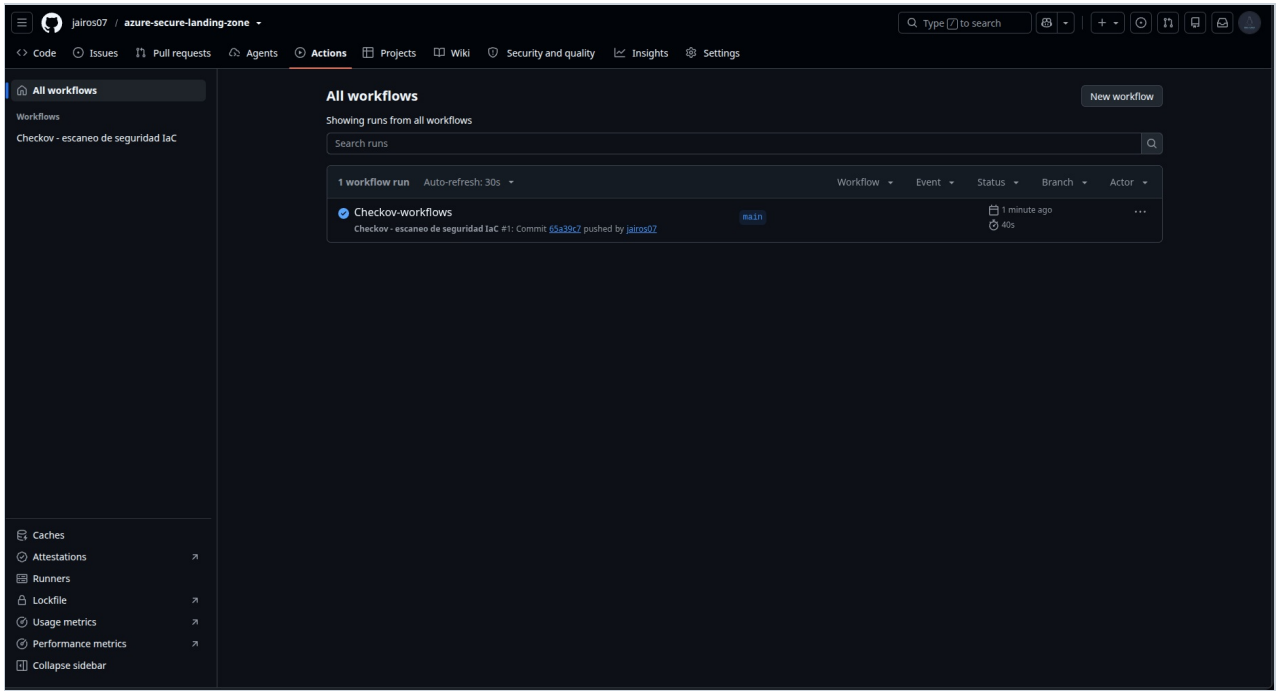
jobs:
  checkov:
    name: Escanear terraform/secure
    runs-on: ubuntu-latest
    steps:
      - name: Descargar el repositorio
        uses: actions/checkout@v4

      - name: Preparar Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      # Misma versión que en local, para resultados iguales
      - name: Instalar Checkov
        run: pip install checkov==3.3.30

      - name: Ejecutar Checkov
        run: checkov -d terraform/secure --compact
```

La carpeta `.github/workflows/` hay que crearla antes (`mkdir -p .github/workflows`), porque `tee` no crea directorios. Resultado de la primera ejecución:



GitHub Actions en verde

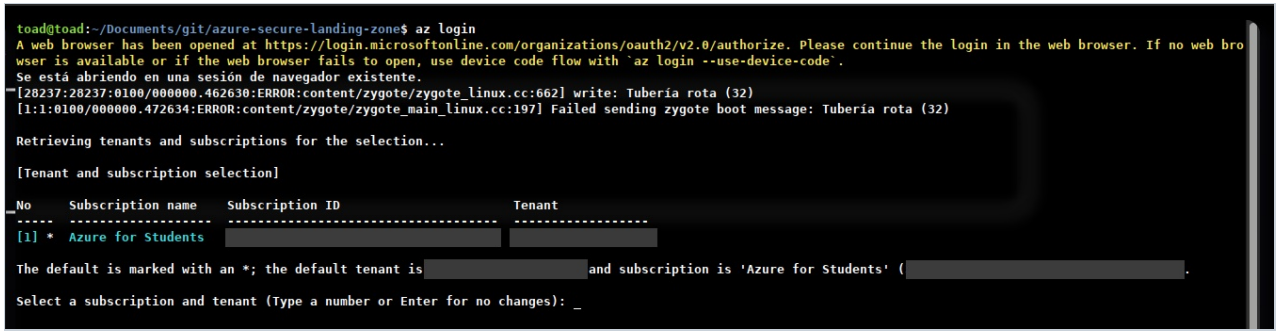
El pipeline **no tiene credenciales de Azure**: solo analiza el código. Por qué no se configuró la conexión sin contraseñas está explicado en el hallazgo 6 (docs/hallazgos.md).

7. Conexión con Azure

7.1 Inicio de sesión

```
az login
```

Abre el navegador y, al terminar, muestra la suscripción disponible. Se guarda una sesión fuera del repositorio.



az login

El ID de suscripción se guarda **solo en la terminal**, nunca en un fichero:

```
export ARM_SUBSCRIPTION_ID=$(az account show --query id -o tsv)
test -n "$ARM_SUBSCRIPTION_ID" && echo "variable OK" || echo "variable VACIA"
```

La variable se pierde al cerrar la terminal, así que hay que repetirla en cada sesión nueva.

7.2 Servicios de Azure necesarios

Como el registro automático está desactivado, se comprueba a mano que están registrados los servicios que usa el proyecto. Cada comando debe devolver `Registered`:

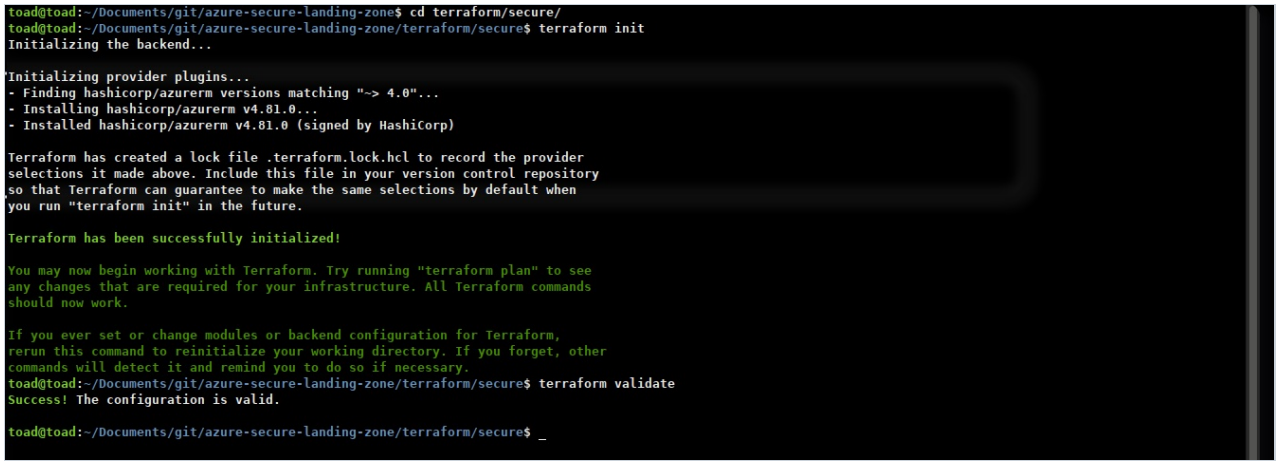
```
az provider show --namespace Microsoft.Resources       --query registrationState -o tsv
az provider show --namespace Microsoft.Network         --query registrationState -o tsv
az provider show --namespace Microsoft.ManagedIdentity --query registrationState -o tsv
```

Si alguno devuelve `NotRegistered`: `az provider register --namespace <nombre>`, esperar uno o dos minutos y repetir.

7.3 Inicialización y validación

```
cd terraform/secure
terraform init
terraform validate
```

`init` descarga el proveedor `azurerm` 4.81.0 y crea `.terraform.lock.hcl`, que sí se sube al repositorio. `validate` debe responder `Success! The configuration is valid.`



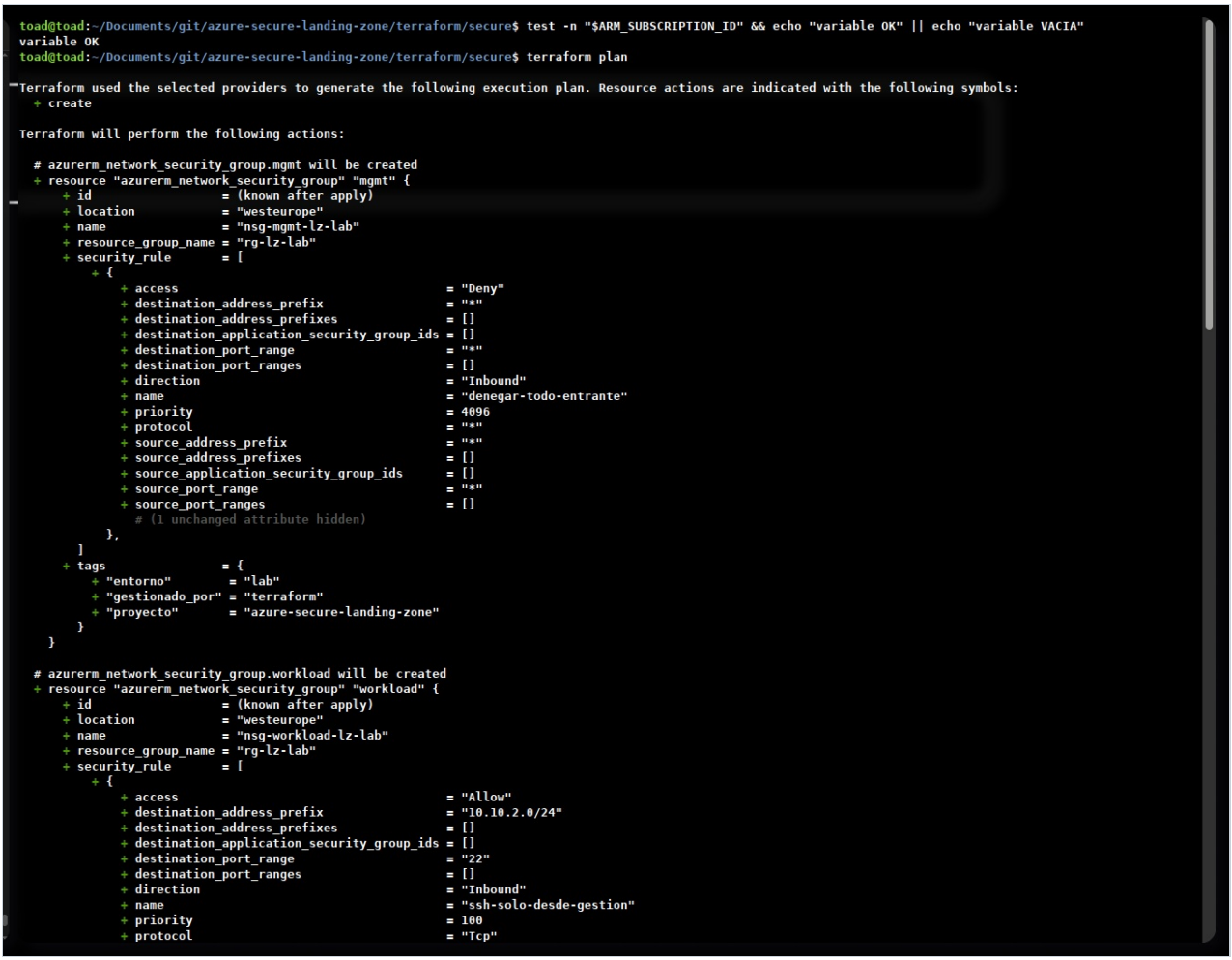
terraform init y validate

8. Plan y despliegue de la red

8.1 Primer plan

```
terraform plan
```

Muestra lo que se crearía **sin crear nada**. El primer plan terminó con `Plan: 8 to add, 0 to change, 0 to destroy.`, con la región por defecto de entonces (`westeurope`):



Plan inicial

8.2 Primer `apply` fallido

```
terraform apply
```

El grupo de recursos se creó, pero la red y los dos NSG fueron rechazados con `RequestDisallowedByAzure`: la suscripción no permite esa región. No se creó nada más y no hubo coste.

8.3 Diagnóstico y corrección

Las regiones permitidas están en una política asignada a la suscripción:

```
az policy assignment list --query "[].{nombre:displayName, regiones:parameters.listOfAllowedLocations.value}" -o json
```

Resultado: la política *Allowed resource deployment regions* permite `norwayeast`, `switzerlandnorth`, `belgiumcentral`, `italynorth` y `germanywestcentral`. Se eligió esta última:

```
sed -i 's/"westeurope"/"germanywestcentral"/' variables.tf
grep -n "location" -A 3 variables.tf
terraform plan
```

Cambiar la región obliga a reemplazar el grupo de recursos (Azure no permite moverlo). Como estaba vacío no se perdió nada, y el plan lo refleja:

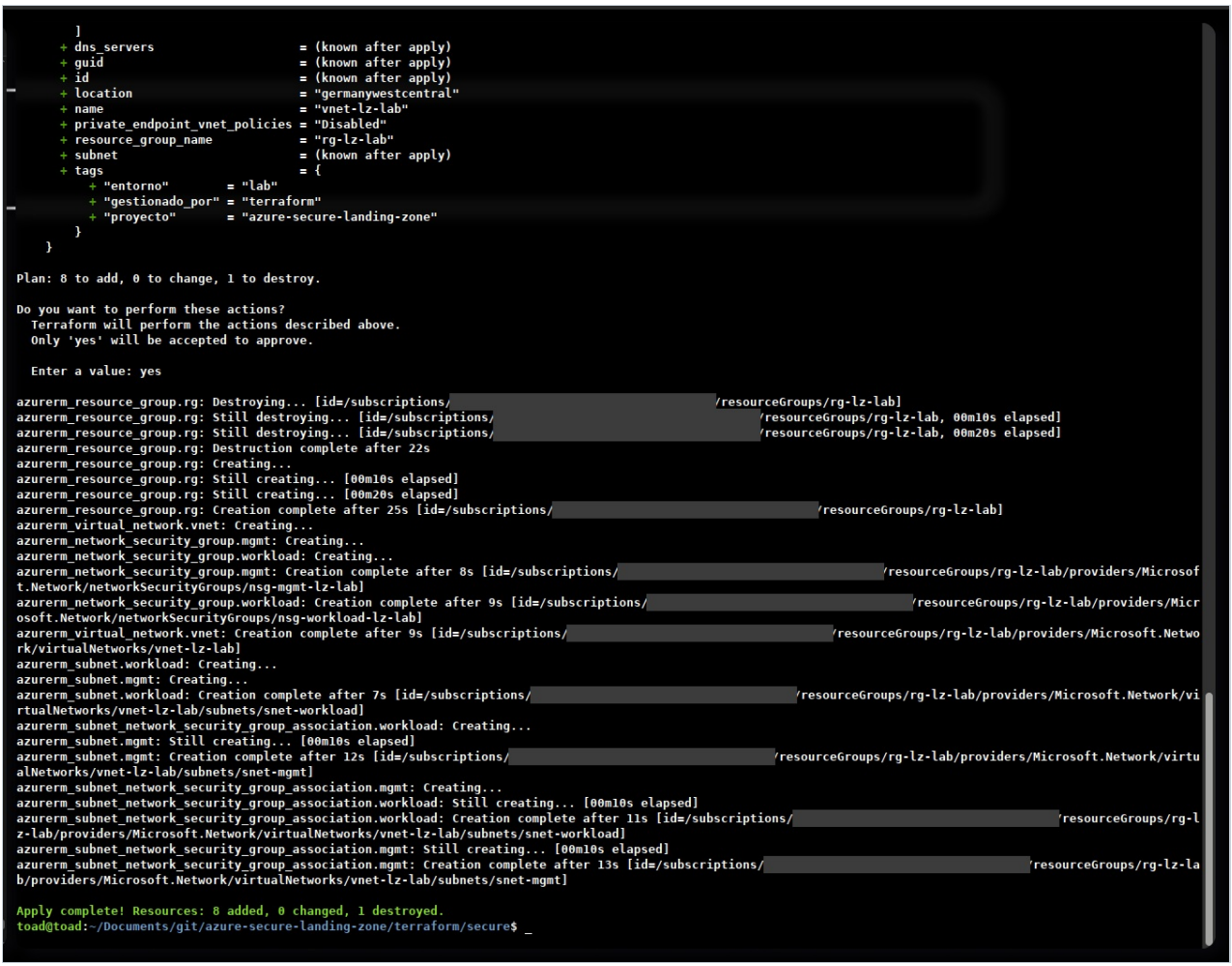
```
Plan: 8 to add, 0 to change, 1 to destroy.
```

8.4 Despliegue

```
terraform apply
```

Se revisa la línea del plan y se escribe `yes`. Resultado:

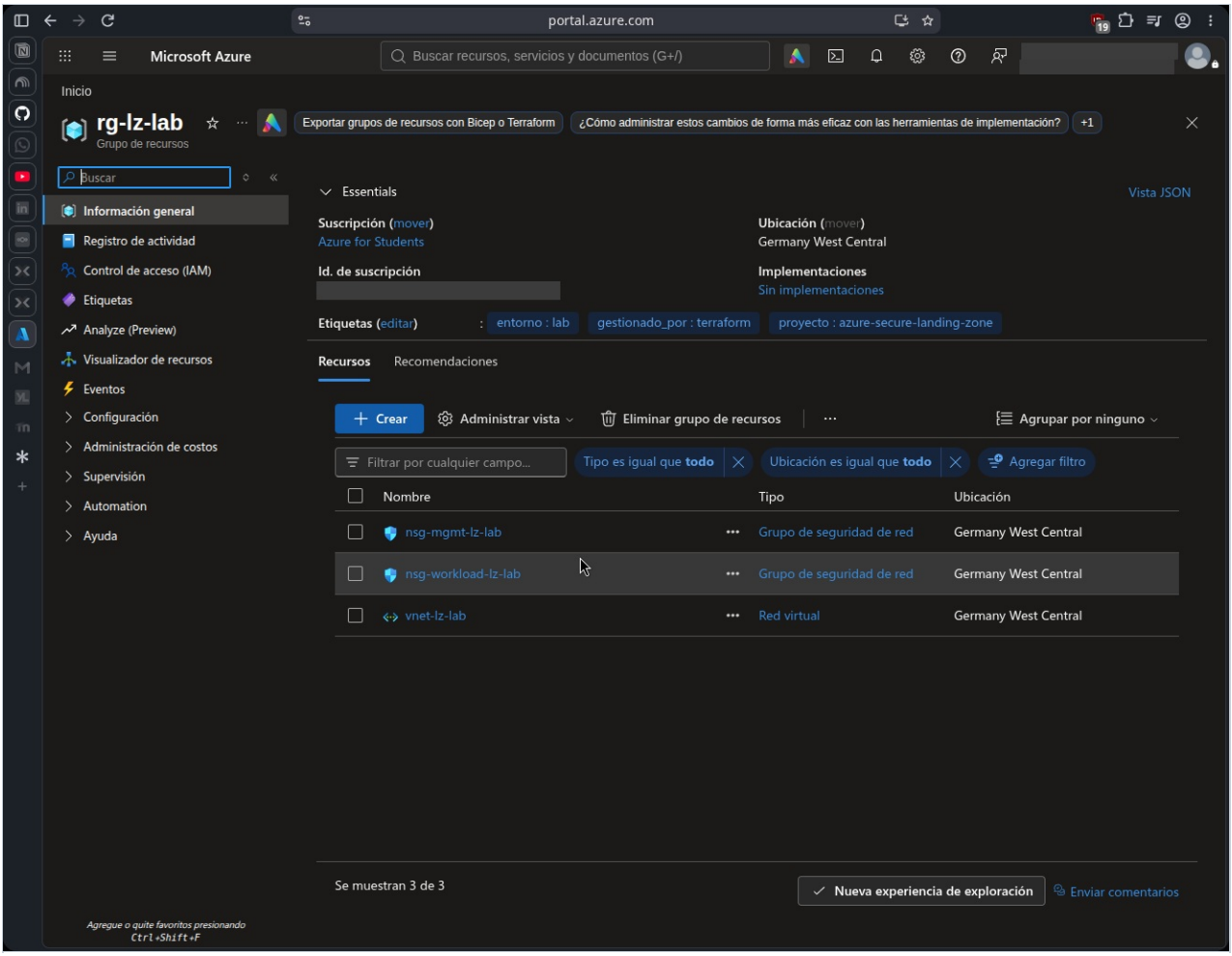
```
Apply complete! Resources: 8 added, 0 changed, 1 destroyed.
```



Apply final

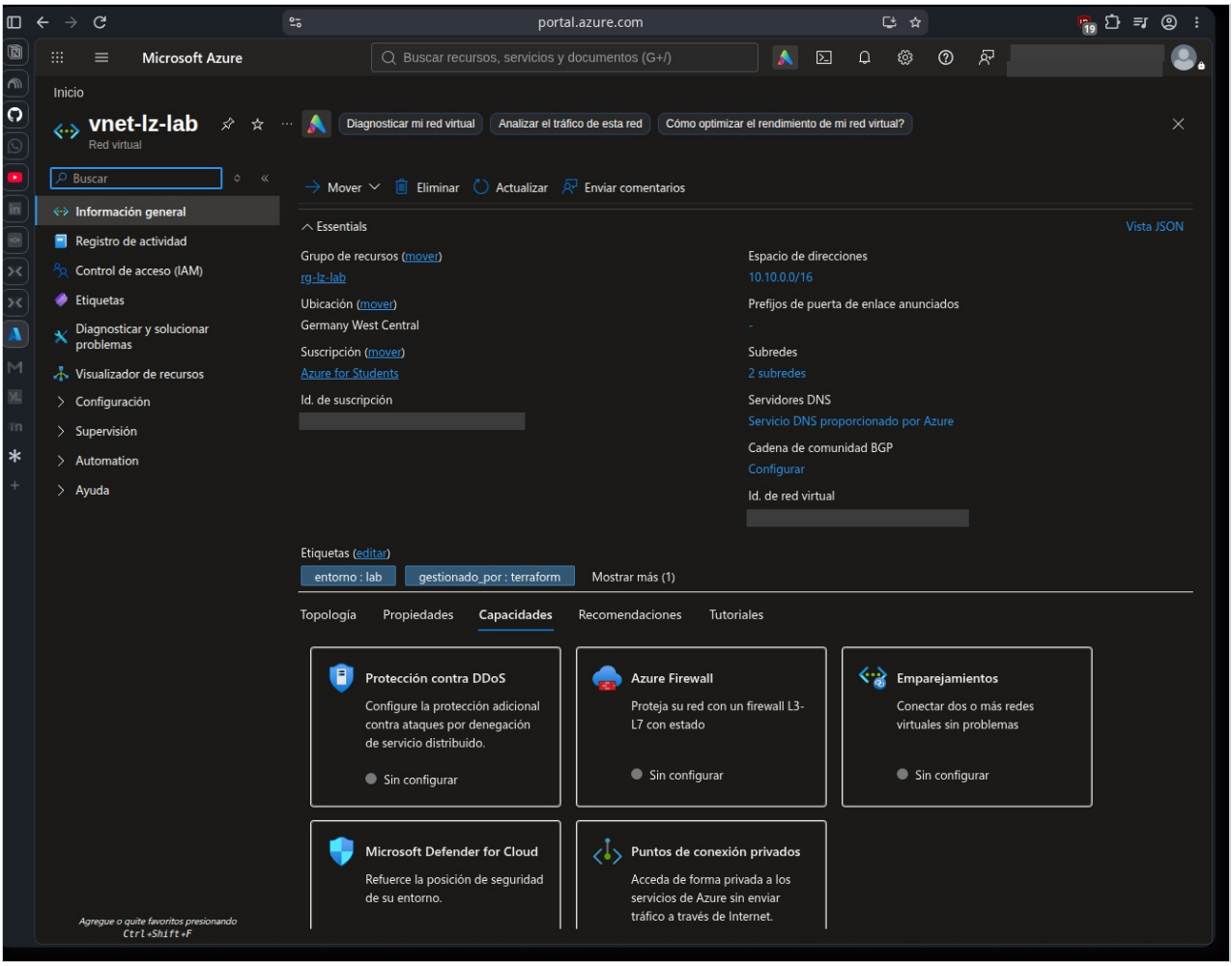
8.5 Comprobación en el portal

Grupo de recursos `rg-lz-lab`. Contiene la red y los dos NSG, con las tres etiquetas.



Grupo de recursos

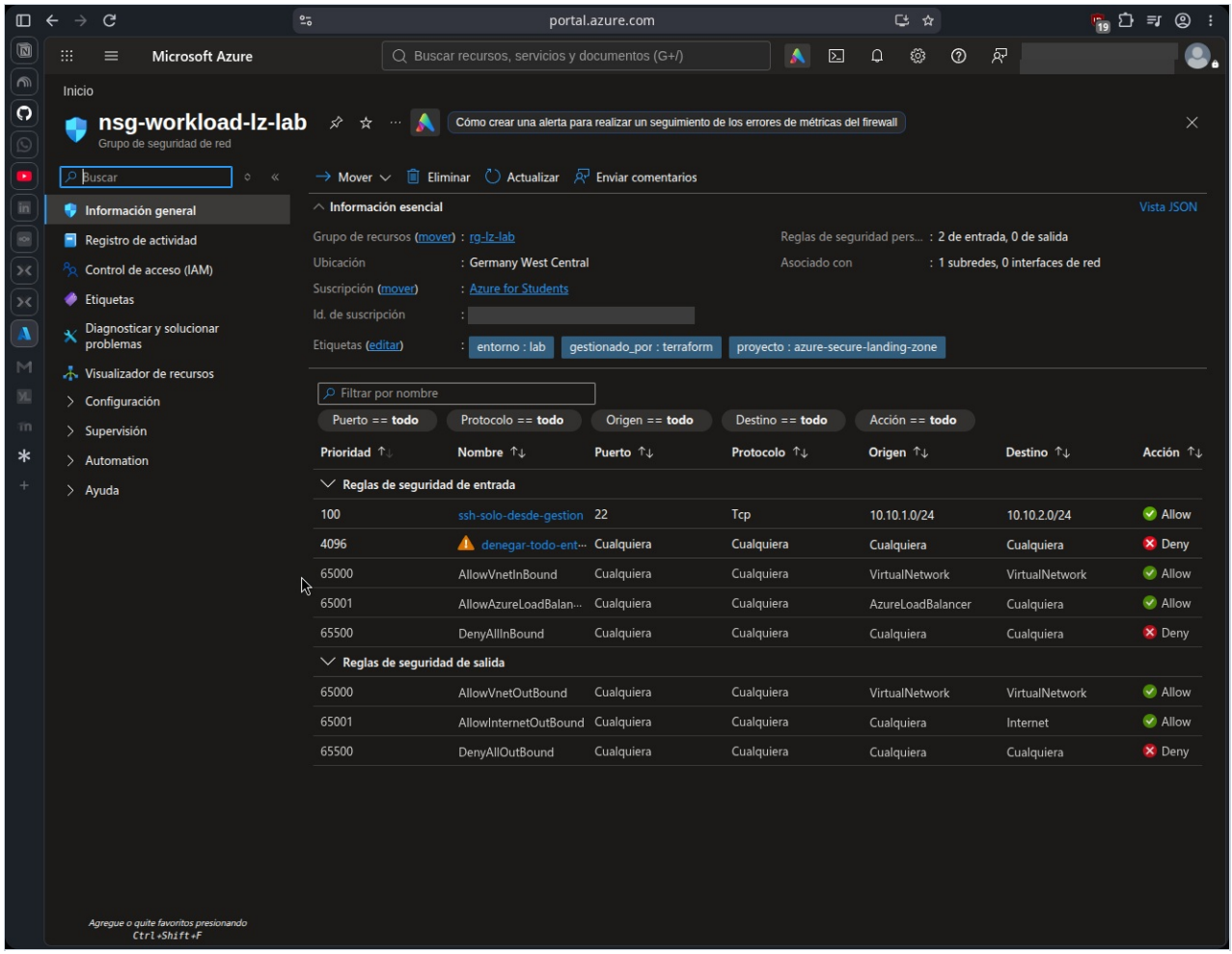
Red virtual `vnet-lz-lab`**.** Espacio `10.10.0.0/16`, dos subredes, región Germany West Central.



Red virtual

NSG `nsg-workload-lz-lab`. Dos reglas propias de entrada y ninguna de salida, asociado a una subred. Las reglas de prioridad 65000 o superior son las que Azure añade por defecto.

Prioridad	Nombre	Acción	Origen	Destino	Puerto
100	`ssh-solo-desde-gestion`	Allow	`10.10.1.0/24`	`10.10.2.0/24`	22/TCP
4096	`denegar-todo-entrante`	Deny	Cualquiera	Cualquiera	Cualquiera



Reglas del NSG

En las reglas de salida por defecto aparece `AllowInternetOutBound`, es decir, la salida a Internet **no** está restringida (ver hallazgo 5 (docs/hallazgos.md)).

9. Azure Policy

9.1 Averiguar los parámetros de la política

En vez de suponer los nombres de los parámetros, se leen de la propia definición:

```
az policy definition show --name 6c112d4e-5bc7-47ae-a041-ea2d9dccd749 --query "{nombre:displayName, tipo:policyType}" -o table
az policy definition show --name 6c112d4e-5bc7-47ae-a041-ea2d9dccd749 --query "parameters" -o json
```

La primera consulta confirma que es *Not allowed resource types* y de tipo `BuiltIn`. La segunda devuelve dos parámetros:

Parámetro	Detalle
`effect`	Valores permitidos `Audit`, `Deny` y `Disabled`. El valor por defecto de la definición es `Deny`
`listOfResourceTypesNotAllowed`	Lista de tipos de recurso prohibidos

9.2 Aplicar en modo auditoría

Con `efecto_politica = "Audit"` se hizo `terraform apply` y se comprobó la asignación:

```
az policy assignment list --resource-group rg-lz-lab --query "[].{nombre:displayName, efecto:parameters.effect.value}" -o table
```

```
Nombre                        Efecto
----------------------------  --------
Prohibir IPs públicas en el lab  Audit
```

9.3 Cambiar a `Deny`

Es una sola palabra, y como el efecto es una variable no se toca el resto del código:

```
sed -i 's|^  default     = "Audit".*|  default     = "Deny"|' variables.tf
terraform plan
terraform apply
```

El plan debe mostrar un único cambio (la asignación de la política). Las políticas tardan entre 15 y 30 minutos en propagarse.

9.4 Prueba de bloqueo

Antes: no debe haber ninguna IP pública en el grupo.

```
az network public-ip list --resource-group rg-lz-lab -o table
```

La lista devolvió vacío. Después se intentó crear una:

```
az network public-ip create --resource-group rg-lz-lab --name pip-prueba-bloqueo \
  --location germanywestcentral --sku Standard --allocation-method Static
```

Resultado, con el identificador de suscripción sustituido (esta prueba no tiene captura, se conserva la salida en texto):

```
(RequestDisallowedByPolicy) Resource 'pip-prueba-bloqueo' was disallowed by policy.
Policy identifiers: '[{"policyAssignment":{"name":"Prohibir IPs públicas en el lab",
"id":"/subscriptions/<ID-DE-SUSCRIPCION>/resourceGroups/rg-lz-lab/providers/
Microsoft.Authorization/policyAssignments/no-ip-publica"},
"policyDefinition":{"name":"Not allowed resource types",
"id":"/providers/Microsoft.Authorization/policyDefinitions/6c112d4e-5bc7-47ae-a041-ea2d9dccd749",
"version":"2.0.0"}}]'.
Code: RequestDisallowedByPolicy
Target: pip-prueba-bloqueo
```

Se repitió el intento y el resultado fue idéntico, así que no fue un efecto de la propagación. Al final `public-ip list` seguía vacío: no se creó nada ni hubo coste.

10. RBAC de mínimo privilegio

10.1 Comprobar los nombres de las acciones

Un nombre de acción mal escrito hace fallar el `apply`, así que se listaron las acciones de lectura que Azure define para red:

```
az provider operation show --namespace Microsoft.Network \
  --query "resourceTypes[?contains(name,'virtualNetworks') || contains(name,'networkSecurityGroups')].operations[].name" \
  -o tsv | grep -i "/read$"
```

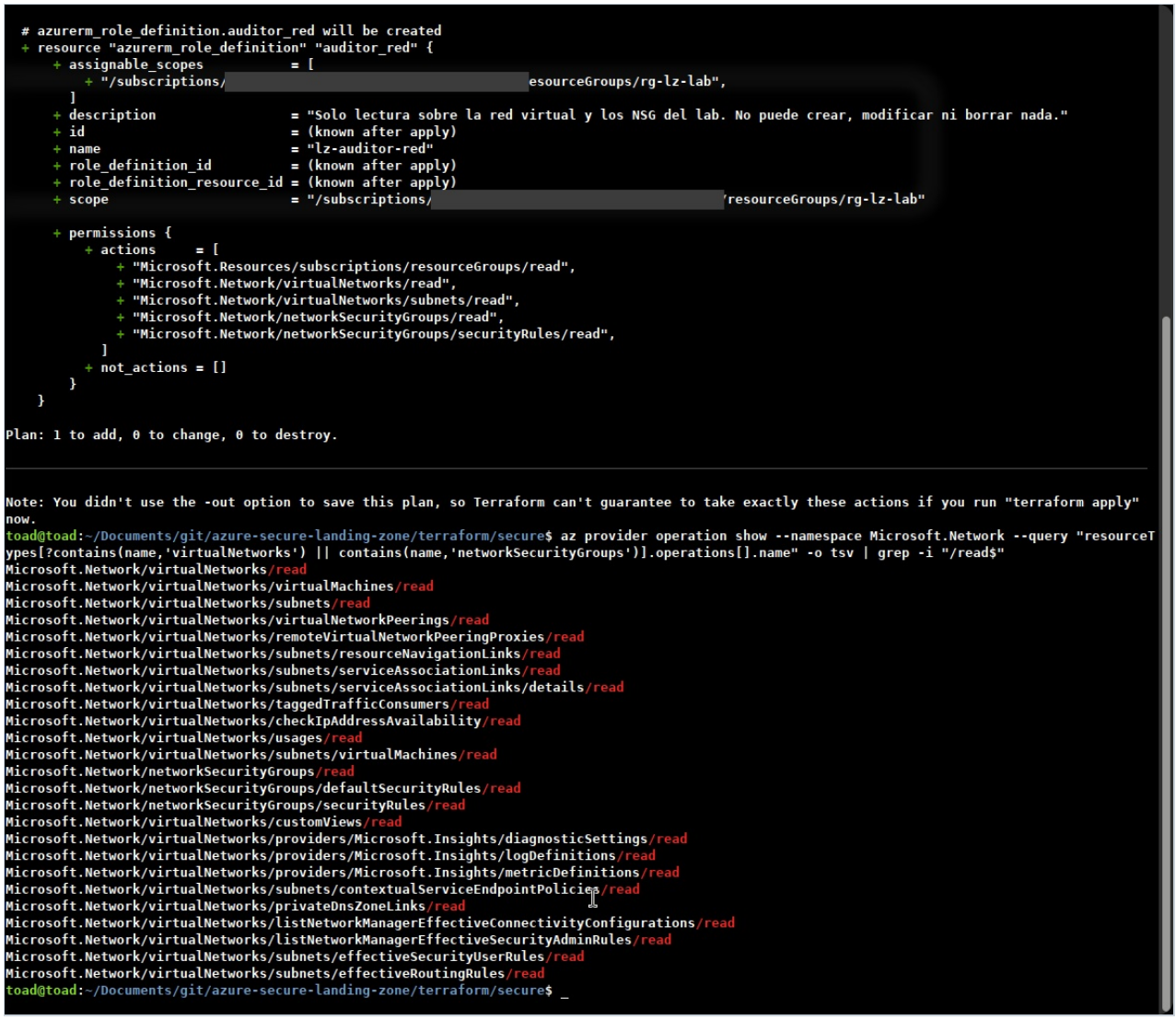
Las cuatro acciones de red que usa el rol aparecen en la lista:

```
Microsoft.Network/virtualNetworks/read
Microsoft.Network/virtualNetworks/subnets/read
Microsoft.Network/networkSecurityGroups/read
Microsoft.Network/networkSecurityGroups/securityRules/read
```

La quinta, `Microsoft.Resources/subscriptions/resourceGroups/read`, permite ver el propio grupo de recursos y no entra en esa consulta.

10.2 Plan y despliegue

Con solo el rol en `rbac.tf`, el plan es `1 to add`. Al añadir la identidad y la asignación, `3 to add`.



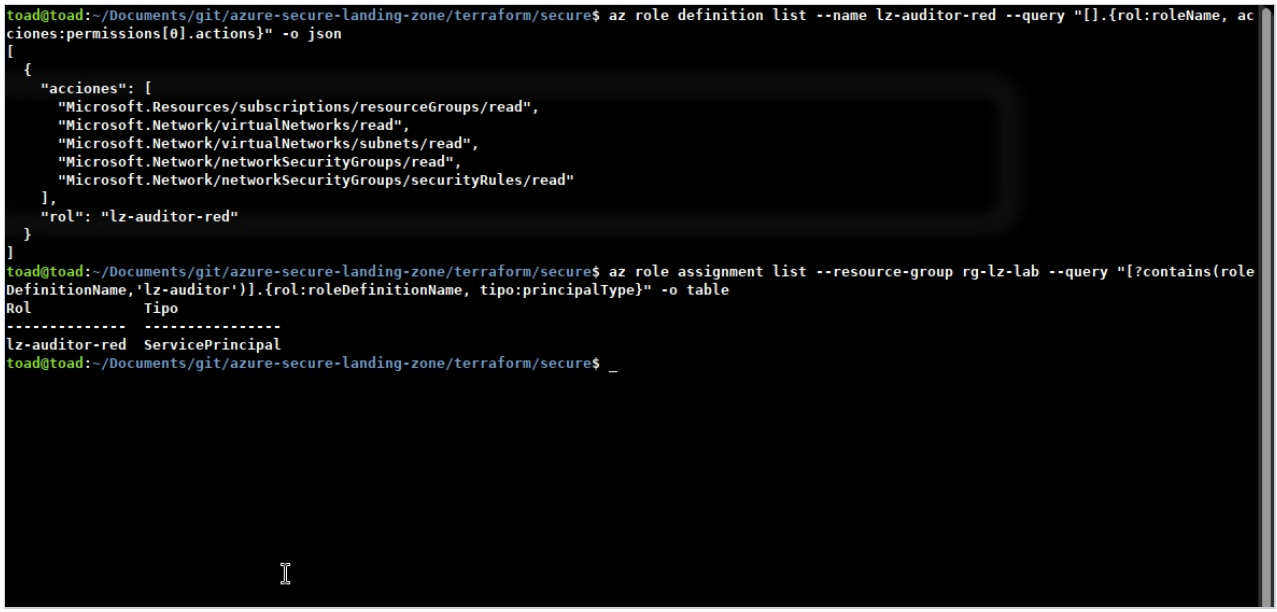
Plan del rol y acciones de lectura

```
terraform validate
terraform plan
terraform apply
```

10.3 Comprobación

```
az role definition list --name lz-auditor-red --query "[].{rol:roleName, acciones:permissions[0].actions}" -o json
az role assignment list --resource-group rg-lz-lab --query "[?contains(roleDefinitionName,'lz-auditor')].{rol:roleDefinitionName,
tipo:principalType}" -o table
```

El rol existe con cinco acciones, todas de lectura, y hay una asignación a una identidad de tipo `ServicePrincipal`:



Rol y asignación

Lo que **no** se ha probado es que la identidad pueda realmente leer lo previsto y no pueda modificar nada (ver `limitaciones.md` (docs/limitaciones.md)).

11. Lista de comprobaciones

Qué se comprueba	Comando	Resultado esperado
El estado queda ignorado por git	`git check-ignore -v terraform/secure/terraform.tfstate terraform/secure/prod.tfvars terraform/secure/.terraform/x`	Tres líneas
Lo que se va a subir	`git status --short`	Solo ficheros de código y documentación
Código válido	`terraform validate`	`Success! The configuration is valid.`
Escaneo de seguridad	`checkov -d terraform/secure --compact --quiet`	`Failed checks: 0`
Sin cambios pendientes	`terraform plan`	Sin cambios, o los esperados
Política activa	`az policy assignment list --resource-group rg-lz-lab --query "[].{nombre:displayName, efecto:parameters.effect.value}" -o table`	Efecto `Deny`
La política bloquea	`az network public-ip create ...`	`RequestDisallowedByPolicy`
Ninguna IP pública	`az network public-ip list --resource-group rg-lz-lab -o table`	Vacío
Rol de solo lectura	`az role definition list --name lz-auditor-red ...`	Cinco acciones terminadas en `read`

12. Limpieza

Para eliminar todo lo desplegado:

```
cd terraform/secure
export ARM_SUBSCRIPTION_ID=$(az account show --query id -o tsv)
terraform destroy
```

Borra el grupo de recursos con su contenido, la política, el rol y la identidad. **No** hay que borrar `terraform.tfstate` antes: Terraform lo necesita para saber qué eliminar. En el momento de redactar este documento no se había ejecutado `destroy` y el entorno seguía desplegado.

Como todo está en código, el entorno se puede volver a crear con `terraform apply`.

13. Problemas encontrados

Síntoma	Causa	Solución
`checkov` termina sin resultados, solo el banner	No hay ficheros `.tf` en la ruta indicada	Ejecutarlo desde la raíz del repositorio y comprobar que los ficheros existen
`tee: No existe el archivo o el directorio`	La carpeta de destino no existe	`mkdir -p` de la carpeta (por ejemplo, `.github/workflows`)
Un fichero acaba en la raíz del repo en vez de en su carpeta	`tee` con ruta relativa ejecutado desde otro directorio	`mv` a `terraform/secure/`
`Error: Missing required provider` en `validate`	Se ejecuta fuera de `terraform/secure/`, donde está el `init`	Entrar en esa carpeta
`plan` sin salida durante mucho tiempo y luego `context canceled`	Registro automático de decenas de servicios, interrumpido con Ctrl+C	`resource_provider_registrations = "none"`
`RequestDisallowedByAzure` al crear la red	Región no permitida por la suscripción	Consultar las regiones permitidas y cambiar `location`
`E: No se ha podido localizar el paquete azure-cli`	Faltaba `apt update` tras añadir el repositorio	`sudo apt update` y repetir la instalación
Errores de firma de otros repositorios al hacer `apt update`	Repositorios ajenos al proyecto con claves o rutas obsoletas	No afectan a la instalación. Se pueden ignorar o arreglar aparte
La consulta de políticas por nombre visible no devuelve nada	Filtro por `displayName` que no coincidía	Consultar directamente por identificador con `--name`
`heredoc` falla en la terminal	fish no admite `<<`	Abrir `bash` antes del bloque

14. Índice de capturas

Fichero	Qué muestra	Sección
`01-checkov-local.png`	Escaneo local, 12 pasadas y 0 fallos	6.1
`02-github-actions-verde.png`	Pipeline de Checkov en verde	6.2
`03-terraform-validate.png`	`init` y `validate` correctos	7.3
`04-plan-inicial-westeurope.png`	Primer plan, con la región que luego se cambió	8.1
`05-apply-final.png`	Despliegue completo en `germanywestcentral`	8.4
`06-portal-grupo-recursos.png`	Grupo de recursos en el portal	8.5
`07-portal-vnet.png`	Red virtual en el portal	8.5
`08-portal-nsg-workload.png`	Reglas del NSG de cargas	8.5
`09-rbac-rol-y-asignacion.png`	Rol y asignación	10.3
`10-terraform-version.png`	Versión de Terraform	4.2
`11-az-login.png`	Inicio de sesión en Azure	7.1
`12-plan-rbac-acciones.png`	Plan del rol y acciones de lectura	10.2